Database Management System

Content:

1.1 Introduction to data, database, Database system, DBMS
1.2 Field, Record, Objects, Primary Key, Alternate Key, Candidate Key
1.3 Advantages of using DBMS
1.4 DDL (Data Definition Language) and DML (Data Manipulation Language)
1.5 Database Model: Network Model, Hierarchical Model, Relational database model
1.6 Concept of Normalization: 1NF, 2NF, 3NF
1.7 Centralized Vs. Distributed Database
1.8 Database Security

1.1 Introduction to data, database, Database system, DBMS

Data: Data means raw facts gathered about someone or something, which is random and unorganized.

Information: Facts which are meaningful is called information. These are organized

Database: The database is a collection of inter-related data which is used to retrieve, insert delete the data efficiently. It is also used to organize the data in the form of table, schema, views, and reports, etc.
Example: The college Database organizes the data about the admin, staff, students and faculty etc.

Database Management System:
DBMS = Database + Management System
A database management system (DBMS) is a software package designed to define, manipulate, retrieve and manage data in a database.
A DBMS generally manipulates the data itself, the data format, field names, record structure and file structure.
For Example: MySQL, Oracle, etc are a very popular commercial database which is used in different applications.

1.2 Field, Record, Objects, Primary Key, Alternate Key, Candidate Key, Foreign Key

1. Field: In a database table, a field represents a single piece of data, such as a name, age, or date. It is the smallest unit of data storage and corresponds to a single attribute of an entity.

2. Record: A record, also known as a row or tuple, is a collection of related fields that represent a single entity within a table. It contains all the data associated with that particular entity.

3. Objects: Each row (record) represents an object in the database.

4. Primary Key: A primary key is a unique identifier for each record in a database table. It ensures that each record can be uniquely identified and accessed quickly. A primary key must have unique values and cannot contain NULL values.

5. Alternate Key: An alternate key, also known as a secondary key, is another candidate key (see below) that could serve as a primary key if the current primary key is unavailable or unsuitable.

6. Candidate Key: A candidate key is a unique key that can uniquely identify each record in a database table. A table may have multiple candidate keys, but one of them is chosen to be the primary key. The primary key is typically the candidate key that is most appropriate and stable for use as a unique identifier.

7. Foreign Key: A foreign key is a field or a set of fields in a table that refers to the primary key of another table. It establishes a link between two tables, defining a parent-child relationship. The foreign key ensures referential integrity, meaning that values in the referencing table must match values in the referenced table's primary key or be NULL.

1.3 Advantages of using DBMS

Database Management Systems (DBMS) offer numerous advantages that make them essential for modern data storage and retrieval. Some of the key advantages of using a DBMS include:

1. Data Integrity and Security: DBMS enforces data integrity rules, such as primary keys, foreign keys, and constraints, to maintain accurate and reliable data. Additionally, it allows for access control and user permissions, safeguarding sensitive information from unauthorized access.

2. Data Sharing: DBMS enables concurrent access to data by multiple users and applications. This allows for data sharing and collaboration across teams and departments, fostering better decision-making and efficiency.

3. Data Organization: DBMS organizes data in a structured manner using tables, rows, and columns. This organization simplifies data retrieval and analysis, making it easier to find and manipulate data as needed.

4. Data Backup and Recovery: DBMS facilitates data backup and recovery mechanisms, ensuring that critical data can be restored in case of hardware failures, data corruption, or other issues.

5. Data Consistency: DBMS enforces ACID (Atomicity, Consistency, Isolation, Durability) properties, ensuring that database transactions are executed in a reliable and consistent manner.

6. Data Scalability: DBMS can handle large volumes of data and can scale vertically (adding more resources to a single server) or horizontally (adding more servers) to accommodate increasing data demands.

7. Data Query and Reporting: DBMS supports querying capabilities, enabling users to retrieve specific data from the database efficiently. Additionally, it allows for generating reports and insights from the data.

8. Reduced Data Redundancy: DBMS minimizes data redundancy through normalization, which optimizes data storage and reduces the chances of inconsistencies and errors.

Overall, DBMS provides a structured and efficient way to manage data, ensuring data accuracy, security, and availability while simplifying data access and manipulation for users and applications.

1.4 DDL (Data Definition Language) and DML (Data Manipulation Language)

DDL (Data Definition Language): DDL is used for defining and managing the structure of the database. It deals with creating, modifying, and deleting database objects like tables, indexes, views, and other schema-related elements. DDL statements do not manipulate data directly; instead, they help in setting up the framework for storing and organizing data.

Common DDL statements include:

- CREATE: Used to create database objects like tables, views, indexes, etc.

- ALTER: Used to modify the structure of existing database objects.
- DROP: Used to remove database objects.
- TRUNCATE: Used to remove all data from a table, but not the table itself.

Example: CREATE TABLE (id INT PRIMARY KEY, name VARCHAR(50), age INT);

DML (Data Manipulation Language): DML is used for interacting with the data stored in the database. It allows you to retrieve, insert, update, and delete data from the database. DML statements operate on the records within the tables, affecting the data itself.

Common DML statements include:

- SELECT: Used to retrieve data from one or more tables.
- INSERT: Used to add new records into a table.
- UPDATE: Used to modify existing records in a table.
- DELETE: Used to remove records from a table.

Example: INSERT INTO users (id, name, age) VALUES (1, 'Krishna', 30);

DDL VS DML

Aspect	**DDL (Data Definition Language)**	**DML (Data Manipulation Language)**
Purpose	Defines and manages database structure	Manipulates data in the database
Operation	CREATE, ALTER, DROP, TRUNCATE, etc.	SELECT, INSERT, UPDATE, DELETE, etc
Focus	Database objects and schema-related tasks	Data stored within the tables
Outcome	Changes database structure	Changes data within the database
Impact	Can impact the entire database schema	Affects specific data records within the tables
Dependency on data	Not directly dependent on existing data	Directly dependent on the data in the database
Example of use cases	Creating tables, indexes, views, etc.	Retrieving, inserting, updating, and deleting data

1.5 Database Model

1. Hierarchical Model:

- Description: The hierarchical model organizes data in a tree-like structure with strict parent-child relationships. Each parent can have multiple children, but each child can have only one parent. This model was prevalent in early database systems, often used for hierarchical data such as file systems.

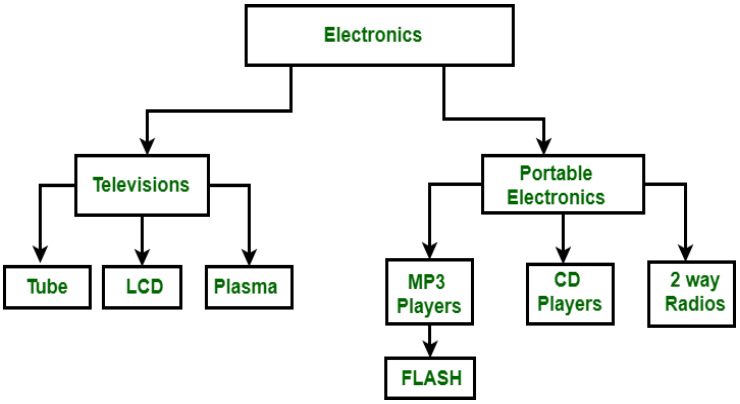
- Advantages:

- Simple and easy to understand, especially for representing one-to-many relationships.
- Efficient for certain types of queries that follow a parent-child traversal pattern.

- Well-suited for applications with clear hierarchical structures, like file systems or organizational charts.

- Disadvantages:

- Limited flexibility due to its rigid tree-like structure, making it challenging to represent complex relationships.
- Data redundancy can occur when multiple entities have similar data under different parent nodes.
- Difficulties in modifying the structure, as changes in the hierarchy might require extensive alterations in the entire database.



2. Network Model:

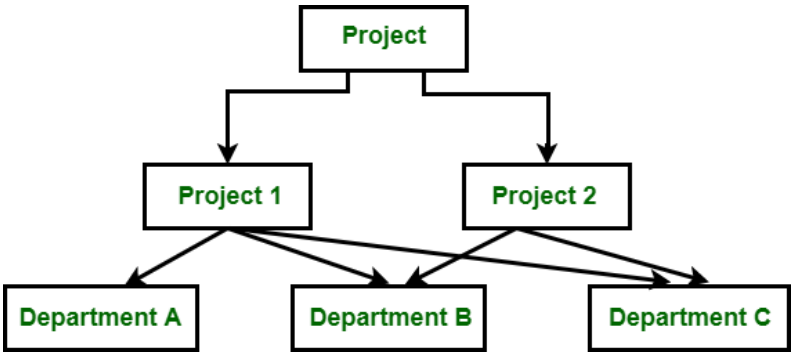
- Description: The network model represents data as nodes interconnected by relationships, forming a graph-like structure. Unlike the hierarchical model, each entity (node) can have multiple connections to other entities. It allows for more complex relationships and many-to-many associations, making it more flexible than the hierarchical model.

- Advantages:

- More flexible than the hierarchical model, allowing multiple parent-child relationships and complex data structures.
- Efficient for modeling many-to-many relationships between entities, such as those found in complex organizational or network scenarios.

- Disadvantages:

- Complexity in implementation and maintenance due to the intricate network of relationships.
- More challenging to query than the hierarchical or relational model, as traversing the graph requires careful navigation.
- Lacks the natural tabular structure and simplicity of the relational model.



3. Relational Model:

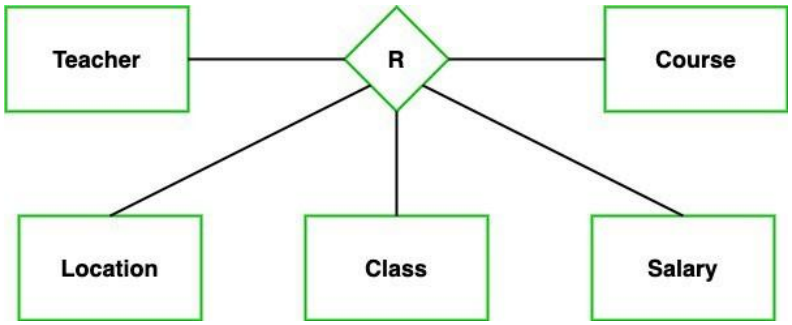
- Description: The relational model organizes data into tables with rows and columns, where each row represents a record (entity) and each column represents an attribute (property). Relationships between tables are established using keys, such as primary keys and foreign keys.

- Advantages:

- Simplicity and ease of understanding with tabular representation of data in tables.
- Provides a clear and straightforward way to establish relationships between tables using keys.
- Powerful querying capabilities through SQL (Structured Query Language), enabling efficient retrieval and manipulation of data.
- Data integrity enforcement through the use of primary keys and foreign keys, ensuring consistent and accurate data.

- Disadvantages:

- Performance may degrade for highly complex queries involving multiple tables or large datasets.
- Data redundancy can occur if not properly normalized, leading to inefficiencies and potential anomalies.
- Lack of support for certain types of data, such as semi-structured or unstructured data, which are better suited for NoSQL databases.



4. Entity-Relationship Model (ER Model):

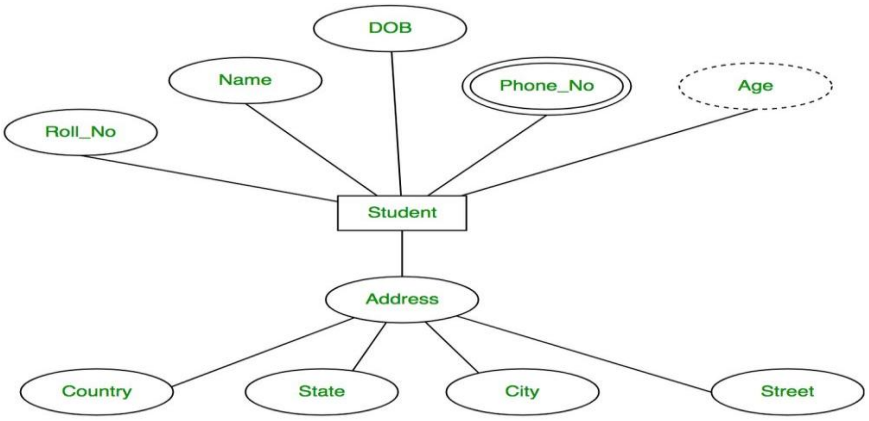
- Description: The Entity-Relationship (ER) Model is a conceptual model that uses entities (objects), attributes (properties), and relationships to represent data requirements. Entities represent real-world objects, attributes define properties of entities, and relationships describe associations between entities.

- Advantages:

- Offers a visual representation of data requirements and relationships, making it easier to communicate database designs to stakeholders and developers.
- Helps in identifying entities, attributes, and relationships more intuitively, aiding the process of database design and planning, especially in complex applications.

- Disadvantages:

- Not as specific as other models in terms of implementation details, requiring translation to a specific database model like the relational model for practical use.
- While useful for conceptual modeling, it may not directly map to the physical implementation of databases, which could add complexity to the development process.



1.6 Concept of Normalization:
Normalization is a systematic approach used in database design to eliminate data redundancy and improve data integrity by organizing data into well-structured and efficient relational tables. It ensures that each piece of data is stored in one place and avoids duplication, making data management more efficient and reducing the chances of data inconsistencies.

The concept of normalization is based on a series of "normal forms," which define rules for structuring data. These normal forms progressively remove specific types of data redundancy and dependency issues. The most common normal forms are:

1. First Normal Form (1NF):
- Eliminate repeating groups by ensuring that each column holds atomic values (i.e., indivisible and not multi-valued).
- Create separate rows for related data, making each cell contain only one piece of data.

Example:

student_id	Name	address	Subject
1	Ram	Butwal	C, C++
2	Hari	Chitwan	Java
3	Krishna	Kathmandu	Python, JS

This table is not in 1NF. Subject's value is not atomic.

student_id	Name	address	Subject
1	Ram	Butwal	C
1	Ram	Butwal	C++
2	Hari	Chitwan	Java
3	Krishna	Kathmandu	Python
3	Krishna	Kathmandu	JS

2. Second Normal Form (2NF):
- Meet 1NF requirements.
- Remove partial dependencies by ensuring that non-key attributes are fully dependent on the entire primary key.

Example:
StudentProject

StudentId	ProjectId	StudentName	ProjectName
S89	P09	Olivia	Geo Location
S76	P07	Jacob	Cluster Exploration
S56	P03	Ava	IoT Devices
S92	P05	Alexandra	Cloud Deployment

As stated, the non-prime attributes i.e. **StudentName** and **ProjectName** should be functionally dependent on part of a candidate key, to be Partial Dependent.
The **StudentName** can be determined by **StudentID**, which makes the relation Partial Dependent.

The **ProjectName** can be determined by **ProjectID**, which makes the relation Partial Dependent.

Therefore, the <**StudentProject**> relation violates the 2NF in Normalization and is considered a bad database design.

To remove Partial Dependency and violation on 2NF, decompose the above tables

<StudentInfo>

StudentID	**ProjectID**	**StudentName**
S89	P09	Olivia
S76	P07	Jacob
S56	P03	Ava
S92	P05	Alexandra

<ProjectInfo>

ProjectID	**ProjectName**
P09	Geo Location
P07	Cluster Exploration
P03	IoT Devices
P05	Cloud Deployment

Now the relation is in 2nd Normal form of Database Normalization

3. Third Normal Form (3NF):

- Meet 2NF requirements.
- Eliminate transitive dependencies by ensuring that non-key attributes depend only on the primary key, not on other non-key attributes.

Advantage:

- Reduced data redundancy, leading to more efficient storage and easier maintenance.
- Improved data integrity and accuracy, as data anomalies are minimized.
- Simplified querying and reporting, as data is organized into logical and meaningful structures.
- Better performance in database operations for most scenarios.

Disadvantage:

- Increased complexity in database design, leading to potentially more complicated queries and joins.
- In some cases, normalization might lead to the creation of numerous smaller tables, impacting query performance.

Example:

Rollno	**Game**	**Feestructure**
1	Basketball	500
2	Basketball	500
3	Basketball	500
4	Cricket	600
5	Cricket	600
6	Cricket	600
7	Tennis	400

The above student table is in 1NF because there are no multivalue attributes.
Student table is also in 2NF because all non-key attributes are fully functional dependent on the primary key (rollno). But the table is not in 3NF because there is transitive dependency i.e. game-> feestructure.

Feestructure has transitive/indirect dependency on rollno via game.

Anomalies

The above student table is also suffering from all three anomalies −

- **Insertion anomaly** − A new game can't be inserted into the table unless we get a student to play that game.
- **Deletion anomaly** − If rollno 7 is deleted from the table we also lost the complete information regarding tennis.
- **Updation anomaly** −To change the fee structure for basketball we need to make changes in more than one place.

So divide the student table into R2(game, feestructure) and R1(rollno, game).

R1

Rollno	**Game**
1	Basketball
2	Basketball
3	Basketball
4	Cricket
5	Cricket
6	Cricket
7	Tennis

R2

Game	**Feestructure**
Basketball	500
Cricket	600
Tennis	400

1.7.1 Centralized Database:

A centralized database is a type of database system where all the data is stored and managed in a single location or server. In this setup, all data processing, storage, and management tasks are handled by a single database server, and all users or applications access the data from this central point. The server acts as a single point of control for data administration and ensures data consistency across the entire system.

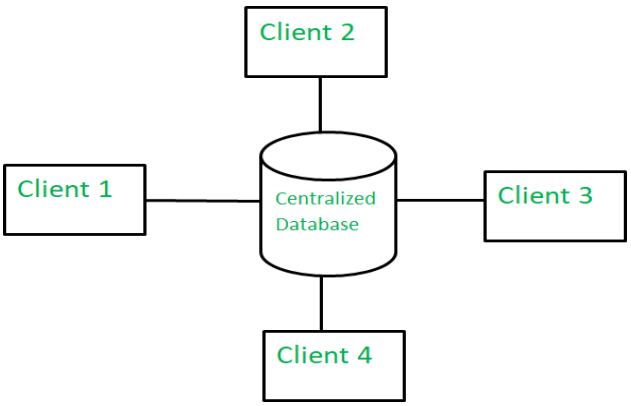


Figure 1: Centralized Database

Advantages of Centralized Database:

- **Simplified data management:** Having all data in one place makes it easier to manage and maintain.
- **Data consistency:** Since all data is stored in a central location, it is easier to enforce data integrity and consistency rules.
- **Lower operational costs:** Centralized databases often require less hardware and infrastructure, leading to potential cost savings.
- **Easier backup and disaster recovery:** Backup and recovery processes can be centralized, making it simpler to ensure data protection.

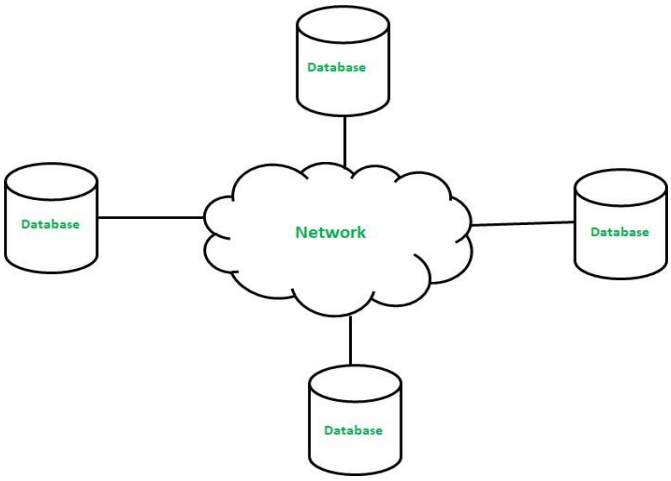
Disadvantages of Centralized Database:

- **Single point of failure:** If the central server experiences a problem, the entire system can be affected.
- **Network dependency:** Accessing data requires a network connection, and performance can suffer if there are network issues.
- **Scalability challenges:** As data and users grow, the central server might become a performance bottleneck.
- **Geographic limitations:** Users located far from the central server may experience higher latency in data access.

1.7.2 Distributed Database:

A distributed database is a type of database system that spreads data across multiple physical locations or database servers, often connected through a network. In this model, data is partitioned and stored in different locations, providing a decentralized approach to data management. Each location may be responsible for its subset of data, and coordination between distributed nodes is required to ensure data consistency and integrity.

Figure 2: Distributed Database



Advantages of Distributed Database:

- **Improved performance:** Data can be located closer to the end-users, reducing network latency and improving data access times.
- **Enhanced scalability:** As data and users increase, additional servers can be added to the distributed system to handle the load.
- **High availability:** If one node fails, data can still be accessible from other nodes, reducing the impact of a single point of failure.
- **Geographic flexibility:** Distributed databases can cater to users in different regions, accommodating diverse geographical requirements.

Disadvantages of Distributed Database:

- **Complexity:** Distributed databases are more challenging to design, implement, and manage due to their distributed nature.
- **Data consistency:** Ensuring data consistency across distributed nodes can be complex and may require sophisticated synchronization techniques.
- **Network overhead:** Communication between distributed nodes can introduce additional overhead and potential performance issues.
- **Security concerns:** The distribution of data raises security challenges, requiring robust access control and encryption measures.

<u>Centralized Database vs Distributed Database</u>

Aspect	**Centralized Database**	**Distributed Database**
Definition	Stores all data in a single location or server.	Spreads data across multiple locations or servers.
Architecture	Data managed in a central server.	Data distributed across multiple nodes.

Data Storage	All Data stored in one database.	Data partitioned and stored in different nodes.
Access	Users and applications access data from the central server.	Data accessed from multiple distributed nodes.
Advantages	Simplified data management and administration.	Improved performance and reduced network latency.
	Data consistency and integrity are easier to enforce.	Enhanced scalability with the addition of nodes.
	Potential cost savings on hardware and infrastructure.	High availability; data remains accessible if one node fails.
	Easier backup and disaster recovery processes.	Geographic flexibility; can cater to users in different regions.
Disadvantages	Single point of failure; impact if central server fails.	Complexity in design, implementation, and maintenance.
	Network dependency; performance affected by network latency.	Ensuring data consistency across distributed nodes is challenging.
	Scalability challenges as data and users grow.	Network overhead and potential performance issues.
	Geographic limitations for users far from the central server.	Security concerns with data distributed across multiple locations.

1.8 Database Security

Database security is a critical aspect of information security that focuses on protecting sensitive data and ensuring the confidentiality, integrity, and availability of databases. As databases hold valuable and sensitive information, it is essential to implement robust security measures to prevent unauthorized access, data breaches, and malicious activities.

1. Access Control:
- Limit access to authorized users through authentication and authorization mechanisms.
- Use role-based access control (RBAC) to assign privileges based on user roles.
- Implement the principle of least privilege to grant minimal necessary access rights.

2. Encryption:
- Encrypt sensitive data at rest and during transmission to protect against unauthorized access.
- Use strong encryption algorithms and manage encryption keys securely.

3. Data Masking:
- Mask sensitive data in non-production environments to maintain data privacy during development and testing.

4. Auditing and Logging:
- Enable audit trails and logs to track database activity and detect security breaches or suspicious actions.
- Monitor and analyze logs for potential threats and anomalies.

5. Regular Updates and Patching:
- Keep the database software up-to-date with the latest security patches to address known vulnerabilities.

6. Secure Coding Practices:
- Develop and use secure coding practices to prevent common vulnerabilities like SQL injection.
- Validate inputs and use parameterized queries to avoid injection risks.

7. Database Activity Monitoring (DAM):
- Use DAM tools to monitor real-time activities and identify abnormal behavior or potential threats.

8. Physical Security:
- Ensure physical security of database servers and storage devices to prevent unauthorized access.

9. Disaster Recovery and Backup:
- Implement robust backup and disaster recovery strategies to protect against data loss and corruption.

10. User Training and Awareness:
- Educate users, administrators, and developers about security best practices and data handling.

Database security is an ongoing process that requires a comprehensive approach to safeguard valuable data and protect against potential risks and cyber threats.

Database Administrator:

DBA stands for Database Administrator. A Database Administrator is a professional responsible for managing, maintaining, and securing databases within an organization. Their primary role is to ensure that databases operate efficiently, reliably, and securely to meet the needs of users and applications.

Key roles and responsibilities of a Database Administrator include:

- **Database Design**: Designing database structures and schemas to ensure data is organized efficiently and meets the requirements of the organization.
- **Database Installation and Configuration:** Installing and configuring database management systems (DBMS) such as MySQL, Oracle, SQL Server, etc., on servers or cloud platforms.
- **Database Maintenance**: Performing routine maintenance tasks such as database backups, updates, and patches to ensure data integrity and system reliability.
- **Performance Tuning:** Optimizing database performance by monitoring and tuning parameters, indexing, and query optimization.
- **Data Security:** Implementing security measures such as access controls, encryption, and authentication to protect sensitive data from unauthorized access or breaches.
- **Backup and Recovery:** Implementing backup and recovery strategies to ensure data is backed up regularly and can be restored in case of data loss or system failure.
- **Database Monitoring:** Monitoring database performance, availability, and resource usage to identify and resolve issues proactively.
- **Capacity Planning:** Planning for future growth and scalability by estimating resource requirements and capacity needs for databases.

- **User Support and Training:** Providing support to users, troubleshooting issues, and offering training on database usage and best practices.

Database Administrators play a critical role in managing the data infrastructure of organizations, ensuring that data is stored, accessed, and managed efficiently and securely to support business operations and decision-making processes.

Assignment Work